



POSTGRADUATE LIFECYCLE MANAGEMENT SYSTEM

Master System Audit, Workflow Reference, and Progress Register

Current implementation baseline • Developer handover • Improvement roadmap

DOCUMENT PGLMS-MASTER-001 • Baseline v1.0

AUDIT DATE 18 July 2026

CODE BASELINE main • 653ff632e0cecb6114f50e6fb525ebe2cb592942

STATUS Current implementation baseline and living progress register

65 API route files	79 / 243 Vitest files / tests passed	34 tracked baseline risks
37 UI page files	24 Prisma models	1 critical dependency-audit severity

Canonical source: docs/PGLMS_MASTER_SYSTEM_AUDIT_AND_PROGRESS_REGISTER.md

Contents

Major sections in the implementation baseline

Document control and maintenance

1. Executive summary

2. Audit scope, method, and limitations
3. System purpose and lifecycle scope
4. Architecture and technology stack
5. Canonical domain model and state machines
6. Observed capability/requirements coverage
7. Roles, responsibilities, and permissions
8. Dashboard and user-interface inventory
9. End-to-end workflow reference
10. Notifications and email trigger catalogue
11. Administrator features, reports, and maintenance
12. API route catalogue
13. Testing, quality, security, and operational readiness
14. Source map for future developers
15. Living progress register
16. Current flaws, risks, and recommended solutions

Use Word's Navigation pane to browse all second- and third-level workflow and risk headings.

Document ID: PGLMS-MASTER-001

Baseline version: 1.0

Audit date: 18 July 2026

Audited branch: main

Audited commit: 653ff632e0cecb6114f50e6fb525ebe2cb592942 — “Implement postgraduate lifecycle workflow updates”

Repository: e23-co2060-MPhil-PhD-Lifecycle-Management-System

Document status: Current implementation baseline and living progress register

Canonical source: This Markdown file. A formatted DOCX is generated from the same baseline.

Source-of-truth rule. This report describes the behavior implemented at the audited commit. It supersedes the workflow claims in docs/WORKFLOW_REPORT.md and output/pdf/pglms_admin_handover_report.* where those documents conflict with the current code. Update the document control, capability matrix, change register, and risk register whenever the system changes.

Document control and maintenance

Field	Current value	Maintenance rule
Baseline owner	Project team	Assign a named product/technical owner after review.
Implementation baseline	Commit 653ff632... on main	Replace only after the revised code is verified.
Requirements baseline	Observable behavior in source, schema, migrations, routes, UI, and tests	Add links to an approved SRS/BRD when one exists.
Verification baseline	npm test and npm run build executed on 18 July 2026	Re-run after every material lifecycle change.
Review cadence	At each accepted change set and before a release	Record the date, owner, evidence, and changed capability/risk IDs.
Change states	Proposed, Approved, In progress, Implemented, Verified, Deferred, Rejected	“Implemented” is not “Verified”; retain evidence for both.
Capability states	Full, Partial, Backend-only, Retired, Missing, Operational verification required	Use the definitions below consistently.

Capability status legend

Status	Meaning
Full	The observed capability has an end-to-end path, role checks, persistence, and usable UI for its current design. It may still have lower-priority

Status	Meaning
	improvement items.
Partial	A meaningful portion exists, but an expected stage, control, notification, UI, or integrity guarantee is incomplete.
Backend-only	Service/API support exists, but no complete user-facing workflow exposes it.
Retired	A route/model/label remains, but the active service intentionally returns 410 Gone or the current migration removed the behavior.
Missing	No implementation path was found.
Operational verification required	The repository references the capability, but deployment/runtime evidence is outside the repository and was not available.

1. Executive summary

PGLMS contains most of the major records and lifecycle stages needed to manage a postgraduate candidate: public application, admission, registration, supervisor assignment, proposal submission, ethics-document submission, progress reports, thesis submission, examiner assignment, viva scheduling, corrections, and graduation/final archive. Four authenticated roles—Student, Supervisor, Examiner, and Administrator—have role-scoped APIs and dashboards. A searchable document repository, email delivery log, in-app notifications, admin reports APIs, test suite, and Sentry integration are also present.

The current version is **not yet a dependable end-to-end lifecycle implementation**. Several features are structurally present but are not usable as complete workflows. The highest-impact findings are:

1. `POST /api/auth/claims` is unauthenticated and can set a Firebase role claim and local database role, including Administrator. This is a critical privilege-escalation path.
2. Examiner review attachments are linked directly to the student and are included in the general repository before an administrator releases the review, creating a high-risk confidentiality breach.
3. Proposal and thesis submissions legitimately allow up to ten files, but their download/version services require exactly one current document. A valid multi-file submission therefore breaks history and current-download workflows.
4. Proposal, progress-report, and thesis examiner-review services exist, but assignment/discovery, source-document access, review forms, release presentation, and notifications are incomplete or absent from the UI.
5. Supervisor sign-off and legacy review panels are retired, but overdue logic, schema fields, route names, labels, and old documentation still assume sign-off. Consequently, every submitted progress report eventually becomes “overdue” with no active way to clear it.
6. Ethics is now document-only. There is no approval/rejection/revision state, yet downstream progress and thesis logic treats any ethics package as clearance.
7. Viva outcome entry is a single examiner-controlled value without per-examiner recommendations, quorum, comments, date enforcement, or administrative ratification. The first examiner can move the thesis to its next status.
8. Several notification events are missing, mislabeled, fire-and-forget, or not retryable. Important transitions—including application rejection, viva outcome, corrections required, and correction approval—can be silent.
9. Upload metadata is often committed before the browser finishes uploading to object storage. A failed upload can leave a valid-looking submission, trigger notifications, and produce a missing file.
10. The dependency audit reported 40 known vulnerabilities in the installed production dependency graph: 1 critical, 11 high, 26 moderate, and 2 low. Direct affected packages include Next.js, Nodemailer, Firebase, Firebase Admin, Sentry, and PostCSS.

Quality gates are encouraging but limited: the production build completed, and all 79 Vitest files / 243 tests passed. The tests largely mock Prisma, Firebase, Supabase Storage, and SMTP. The four Playwright files are not part of `npm test`, and no live end-to-end environment, CI pipeline, deployment configuration, or runtime security-header evidence is present in the repository.

Baseline verdict

Area	Verdict
Major lifecycle records	Mostly implemented
Role-scoped API access	Broadly implemented, with one critical claims bypass and several policy inconsistencies
Role responsibilities	Partially aligned; Supervisor and Examiner responsibilities are substantially narrower in the UI than backend/domain names imply
Dashboards	Functional KPI/link launchers; metrics and destinations contain several misleading or dead states
Workflow completeness	Partial; admissions and basic submissions are strongest, while formal review/release/finalization governance is incomplete
Notifications/email	Partial; many triggers work, but coverage, taxonomy, durability, retry, and deep links are incomplete
Document handling	Partial; signed storage is present, but versioning, upload finalization, review confidentiality, and content verification need redesign
Production readiness	Not established by repository evidence

2. Audit scope, method, and limitations

2.1 What was inspected

The audit covered:

- Prisma schema, 24 models, 12 enums, and all migrations;
- 65 API route files and their HTTP methods, authentication wrappers, service calls, and retired behavior;
- 37 page routes, shared layouts, 57 components, role navigation, dashboards, forms, tables, loading/error/empty states, and accessibility indicators;
- lifecycle services for applications, registration, proposals, ethics, progress reports, examiner reviews, theses, viva, corrections, documents, archives, reports, notifications, email, Firebase, and Supabase Storage;
- all 83 test/spec files: 79 Vitest unit/integration-style files and 4 Playwright files;
- project scripts, dependencies, environment-variable usage, Sentry/Next configuration, Git state, and existing documentation;
- a production build, full Vitest run, targeted workflow tests, and a current `npm audit --omit=dev` snapshot.

2.2 Evidence collected

Check	Result
<code>npm test</code>	Passed: 79 files, 243 tests
<code>npm run build</code>	Passed: Prisma generation and Next.js production build completed; 69 routes/pages generated
Targeted backend test set	Passed: 46 tests across 21 files
Repository state before report	Clean <code>main</code> worktree at audited commit
API route files	65
UI page files	37
Component files	57
Prisma models/enums	24 / 12
Dependency audit	40 findings: 1 critical, 11 high, 26 moderate, 2 low

2.3 Limitations and interpretation

No approved external Software Requirements Specification, Business Requirements Document, or acceptance-test catalogue was found. A few source comments refer to REQ-FN-018, REQ-FN-019, and REQ-FN-020, but they are not a complete requirement set. Therefore, “fulfilled requirement” in this report means **an observable capability implemented by the audited repository**, not formal contractual acceptance.

The audit did not have production data, a live Firebase project, live Supabase bucket, SMTP inbox, deployed headers, scheduler, backup policy, or authenticated browser accounts. Runtime-only behavior is explicitly marked for operational verification. No application code was modified during the audit.

3. System purpose and lifecycle scope

PGLMS is a postgraduate lifecycle management web application intended to centralize the journey from prospective applicant to final thesis archive. Despite the repository name and product branding, the implementation accepts four programme types: MPhil, PhD, MSc, and MEng (prisma/schema.prisma:17-22).

The implemented lifecycle is:

```
Public application
→ administrator review/admission
→ student account + active registration
→ supervisor assignment
→ proposal submission/decision
→ ethics evidence submission
→ progress reporting and optional examiner review
→ thesis submission
→ examiner assignment and optional examiner review
→ viva scheduling/outcome
→ corrections when required
→ administrator finalization/graduation
→ record/document retention
```

Not every arrow is enforced. In particular, examiner reviews are not mandatory before decisions or viva, ethics has no decision, and some generic status endpoints can bypass specialized workflows.

4. Architecture and technology stack

4.1 Logical architecture

Layer	Current implementation	Notes
Web/UI	Next.js App Router, React, TypeScript, Tailwind CSS, shadcn/Radix components	Server and client components; role dashboards under <code>/dashboard</code> .
API	Next.js Route Handlers under <code>src/app/api</code>	JSON APIs; most protected handlers use <code>withAuth</code> .
Domain/services	TypeScript modules under <code>src/lib</code>	Lifecycle rules, validation, notifications, storage paths, reports.
Authentication	Firebase Authentication and Firebase Admin session cookies/custom role claims	Seven-day absolute session, 30-minute inactivity window.
Authorization	Firebase role claim plus local active-user lookup; resource checks in services	Database role is not compared on each request.
Database	PostgreSQL accessed through Prisma; repository configuration targets Supabase	Domain records and notification logs.
File storage	Supabase Storage using service-role credentials and 15-minute signed URLs	Client-to-storage upload is common.
Email	Nodemailer over SMTP	Every attempted email is intended to create <code>NotificationLog</code> .
In-app notifications	<code>Notification</code> table and sidebar drawer	Recent eight items only; bulk mark-all-read.
Monitoring	Sentry Next.js integration	Production browser source maps enabled; runtime configuration requires verification.
Testing	Vitest, Testing Library, jsdom, Playwright	Playwright is not wired into the standard test script.

4.2 Dependency versions at the audited commit

Purpose	Package/version
Framework	Next.js 14.2.28
UI runtime	React / React DOM 18.3.1
Language	TypeScript 5.8.3
ORM	Prisma / <code>@prisma/client</code> 5.22.0
Authentication	Firebase 10.14.1; Firebase Admin 12.7.0

Purpose	Package/version
Storage client	@supabase/supabase-js 2.105.x
Validation	Zod 3.24.1
Email	Nodemailer 6.10.1
Data fetching	SWR 2.3.3
Styling/components	Tailwind 3.4.17, Radix, shadcn, Lucide
Monitoring	@sentry/nextjs 10.51.x
Unit/integration-style tests	Vitest 2.1.8, Testing Library
Browser tests	Playwright 1.51.1

Source: package.json:5-64.

4.3 Configuration inventory

Concern	Environment keys referenced by code
Database	DATABASE_URL (Prisma deployment configuration)
Supabase	SUPABASE_URL or NEXT_PUBLIC_SUPABASE_URL; SUPABASE_SERVICE_ROLE_KEY; SUPABASE_STORAGE_BUCKET or public fallback
Firebase Admin	FIREBASE_PROJECT_ID, FIREBASE_CLIENT_EMAIL, FIREBASE_PRIVATE_KEY
Firebase browser app	NEXT_PUBLIC_FIREBASE_API_KEY, auth domain, project ID, storage bucket, sender ID, app ID, measurement ID, with non-public fallbacks in the client module
SMTP	SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS, optional SMTP_FROM
Application links	APP_BASE_URL
Session naming	Optional SESSION_COOKIE_NAME, SESSION_ACTIVITY_COOKIE_NAME
Maintenance	Optional CRON_SECRET; optional PROGRESS_REPORT_OVERDUE_AFTER_DAYS
Sentry	SENTRY_ORG, SENTRY_PROJECT, and platform-provided Sentry configuration

Environment files are ignored, but the repository provides no .env.example; onboarding must be derived from the README and code (.gitignore:26-30).

4.4 Deployment and operations status

- npm run build runs prisma generate && next build; npm start runs next start (package.json:5-14).

- No Dockerfile, Vercel configuration, Kubernetes manifests, release workflow, environment template, or database-deployment pipeline was found.
- `.github/workflows/sync.yml` is empty; there is no working CI quality gate.
- No scheduler definition was found for registration/progress maintenance.
- No backup, restore, disaster-recovery, retention, or data-deletion policy is represented in the repository.
- Security headers are not set in `next.config.mjs` or middleware. They may exist at a deployment edge and must be verified at runtime.
- `next.config.mjs:4-6` sets `productionBrowserSourceMaps: true`; confirm that source maps are uploaded privately and removed from the public artifact.

5. Canonical domain model and state machines

5.1 Core records

Record	Role in the lifecycle	Key implementation note
User	Identity, role, activation	One optional role profile; local role can drift from Firebase claim.
Student	Candidate, programme, academic state	Has <code>isArchived</code> and academic status in addition to user activation.
Application	Public admission application	Applicant is not a <code>User</code> until admission.
Registration	Time-bounded enrolment period	Multiple historical rows; no DB guarantee of exactly one active row.
ResearchProposal	Current proposal metadata/status	Logical versions are inferred from <code>Document.version</code> .
EvaluationForm	Proposal examiner feedback	No proposal examiner assignment; release fields have no active update workflow.
EthicsApproval	Submitted ethics document package	Despite its name, it has no decision/status in the current schema.
SupervisorAssignment	Primary/co-supervisor link	Maximum and primary rules exist only in application code.
ProgressReport	Period narrative/report	Legacy sign-off/overdue fields remain.
ProgressReportReview	New examiner review for a report	Backend-only; review release is not surfaced.
ReviewPanel and related models	Legacy review-panel design	Routes/services are retired, but schema remains.
Thesis	Thesis metadata and state	File versioning is represented by document rows.
ThesisExaminerAssignment	Examiner assignment and review	Assignment also stores review/release data.
Viva	Single scheduled viva and shared outcome	One outcome for all examiners, no per-examiner recommendation.
CorrectionDocument	Correction package and admin approval	No rejection state or decision note.
Document	Polymorphic stored-file metadata	Multiple nullable parents; no DB check that exactly one parent is set.
Notification	In-app user message	No action URL or individual-read API.
NotificationLog	Email-delivery/download audit record	Event taxonomy is frequently reused

Record	Role in the lifecycle	Key implementation note
		incorrectly.

Model and enum definitions: `prisma/schema.prisma:10-557`.

5.2 Implemented state transitions

Application

```
SUBMITTED → UNDER_REVIEW | ADMITTED | REJECTED
UNDER_REVIEW → ADMITTED | REJECTED
ADMITTED → terminal
REJECTED → terminal
```

Repeating the current state is accepted. Source: `src/lib/prisma/application-status.ts:3-31`.

Proposal

```
SUBMITTED → UNDER_REVIEW | APPROVED | REJECTED
UNDER_REVIEW → APPROVED | REJECTED
REJECTED → SUBMITTED
APPROVED → terminal
```

A rejected proposal can be revised. Source: `src/lib/prisma/proposal-status.ts:3-39`.

Registration

```
ACTIVE + expired → LAPSED
renew any registration → archive current ACTIVE if present + create new ACTIVE period
```

Renewal is not an approval workflow. Source: `src/lib/registrations.ts:66-80,147-229`.

Thesis advertised transition map

```
SUBMITTED → UNDER_EXAMINATION
UNDER_EXAMINATION → CORRECTIONS_REQUIRED | FINAL_ARCHIVE
CORRECTIONS_REQUIRED → SUBMITTED | FINAL_ARCHIVE
FINAL_ARCHIVE → CLOSED
CLOSED → terminal
```

Source: `src/lib/prisma/thesis-status.ts:3-28`. Viva FAIL directly sets CLOSED, bypassing this map.

Viva outcome mapping

```
PASS → FINAL_ARCHIVE
MINOR_CORRECTIONS → CORRECTIONS_REQUIRED
MAJOR_CORRECTIONS → CORRECTIONS_REQUIRED
FAIL → CLOSED
```

Source: `src/lib/vivas.ts:115-149,192-255`.

Student academic state

ACTIVE, UNDER_REVIEW, GRADUATED, and ARCHIVED exist, but transitions are not centralized. Specialized thesis finalization sets GRADUATED; record archival sets ARCHIVED; generic thesis status changes can leave them inconsistent.

5.3 State consistency warning

“Archive” currently has four different meanings:

11. `RegistrationStatus.ARCHIVED` — a superseded registration period;
12. `Student.isArchived / AcademicStatus.ARCHIVED` — administrative record retention;
13. `ThesisStatus.FINAL_ARCHIVE` — successful academic finalization stage;
14. `Thesis.isArchived` — a soft-archive flag that current finalization does not set.

These must not be treated as interchangeable.

6. Observed capability/requirements coverage

ID	Observable capability	Current status	Evidence/qualification
CAP-001	Public multi-step programme application	Full for the current basic design	Four steps, validation, PDF/ZIP evidence, success page. No save/resume or applicant tracking.
CAP-002	Administrator application review and admission	Partial	Status transitions and admission provisioning work; under-review UI and applicant rejection notification are missing.
CAP-003	Student/Firebase account provisioning	Partial	Account, claim, profile, and registration created; insecure temporary-password delivery and collision handling remain.
CAP-004	Registration period and renewal	Partial	Active/lapsed/archive periods and renewal work; no approval, payment/evidence, UI, idempotency, or graduated-user blocking.
CAP-005	Supervisor assignment	Partial	Admin create/promote/remove with limits; concurrency, history, removal notices, and student notices are missing.
CAP-006	Proposal submission, revision, and admin decision	Partial	Core flow exists; multi-file versions break, admin evidence is incomplete, and examiner review is not required.
CAP-007	Assigned proposal examiner review	Missing/partial backend	Review endpoint exists without assignment, queue, usable source-document access, or UI.
CAP-008	Formal ethics approval	Missing under current design	Ethics is a document package only; decision route returns 410.
CAP-009	Progress-report submission and supervisor monitoring	Partial	Submission/history/monitoring exist; upload finalization and overdue meaning are flawed.
CAP-010	Supervisor progress-report sign-off	Retired	Both sign-off paths return 410; labels and fields remain.
CAP-011	Progress review panels	Retired	Legacy panel endpoints return 410; schema remains.
CAP-012	Examiner progress-report review and admin release	Backend-only	Create/submit/release services exist; no usable queue/UI/notifications/student release view.
CAP-013	Thesis submission and revision	Partial	Preconditions and submission exist; ethics is not approved, multi-file

ID	Observable capability	Current status	Evidence/qualification
			versions break, upload finalize is absent.
CAP-014	Thesis examiner assignment	Partial	Admin can assign and email examiner; link expires quickly, no removal/reassignment/count rule, limited notification.
CAP-015	Thesis examiner report and admin release	Backend-only	Submit/release API exists; no UI, no notifications, no student retrieval of released review.
CAP-016	Viva scheduling	Full for a simple single-viva design	Future date, venue, upsert, student/examiner notifications. No calendar integration or review prerequisites.
CAP-017	Governed viva decision	Partial/high risk	One examiner sets one shared outcome; no quorum, report, comments, date check, or ratification.
CAP-018	Correction submission and approval	Partial	Student package and admin approval exist; type mismatch, rejection, review notes, download UI, and notices are missing.
CAP-019	Final archive and graduation	Partial	Specialized action graduates and notifies; status/archive/registration/account consistency is incomplete.
CAP-020	Administrative record archival	Partial	Soft-archives selected records; active registrations, theses, documents, assignments, account, and Firebase access remain.
CAP-021	Role-scoped document repository	Partial	Search/filter/signed download and admin soft delete exist; review confidentiality and thesis supervisor policy conflict remain.
CAP-022	In-app notification centre	Partial	Recent items and mark-all-read work; badge, unread total, refresh, deep links, pagination, and individual read are missing.
CAP-023	Email delivery and audit log	Partial	SMTP send and SENT/FAILED log exist; taxonomy, retry, durability, HTML escaping, and complete coverage are missing.
CAP-024	Role dashboards	Partial	Responsive KPI/action shell exists; metrics, dead links, profile identity, drill-downs, trends, and reports need work.

ID	Observable capability	Current status	Evidence/qualification
CAP-025	Administrator operational reports	Backend-only	Student, graduation, thesis-pipeline, overdue, under-review, and archive APIs have no dashboard pages.
CAP-026	Lifecycle/audit history	Missing	Status fields overwrite history; only email attempts and a subset of downloads are logged.
CAP-027	Automated maintenance	Operational verification required/partial	Endpoint exists; scheduler absent, secret fail-open, reminders not deduplicated.
CAP-028	Production monitoring	Operational verification required	Sentry wrapper exists, but deployed DSN, alerts, dashboards, and source-map exposure were not verified.
CAP-029	Automated quality gates	Partial	Unit/integration-style tests and build pass; no effective CI, live integration suite, or standard E2E script.
CAP-030	Accessibility/responsive baseline	Partial	Good component primitives and responsive grids; landmarks, labels, reduced motion, live regions, zoom, and mobile behavior need verification/fixes.

7. Roles, responsibilities, and permissions

7.1 Role responsibility summary

Actor	Responsibilities currently exposed	Responsibilities implied but not fully supported
Public applicant	Complete application, upload/remove draft documents, submit	Save/resume, reference number, status tracking, receive under-review/rejection notices
Student	Maintain own profile fields, submit proposal/ethics/progress/thesis/corrections, view own lifecycle/documents/notifications, renew registration through API	Request-based renewal, see released examiner feedback, receive all decisions, control staged uploads, formally acknowledge requirements
Supervisor	View assigned roster/profile, monitor proposal state and progress-report narratives/documents, browse assigned-student repository	Proposal evaluation, progress sign-off, review comments, milestone approval; these are retired or not implemented despite route names
Examiner	View assigned thesis documents/vivas, record viva outcome; backend can submit proposal/progress/thesis reviews in limited ways	Discover assigned review work, access proposal/progress evidence, submit reports in UI, follow corrections, provide independent viva recommendation
Administrator	Manage users, applications, status decisions, assignments, viva, corrections/finalization, documents, notification log; access report APIs	Evidence-first decision queues, review release UI, report dashboards, retry failed messages, consistent audit trail, safe claims management

7.2 Permission matrix

Legend: **C** create/submit; **R** read/list/download; **U** update/decide; **A** administrative management; **—** no intended access; **B** backend-only/limited.

Domain	Applicant	Student	Supervisor	Examiner	Administrator
Application draft/upload/submit	C/U	—	—	—	R/U decision
User accounts	—	own session	own session	own session	C/R/deactivate
Registration	—	R/U renew own (API only)	R via roster	—	R/U renew
Supervisor assignment	—	R indirectly	R own assignment	—	C/R/U/delete
Proposal	—	C/R revisions	R assigned student	B review without assignment; source	R/U status

Domain	Applicant	Student	Supervisor	Examiner	Administrator
				files denied	
Ethics package	—	C/R	R through repository if assigned	—	R/download; no decision
Progress reports	—	C/R	R assigned, view-only	B assigned review submit	B assign/release; report monitoring
Thesis	—	C/R own	Repository can expose assigned-student thesis despite specialized denial	R assigned	R/U status
Thesis examiner assignment	—	—	—	R own assignment	C/R
Thesis examiner review	—	no release view	—	B submit via API	B release via API
Viva	—	notification/progress only	—	R assigned/U shared outcome	C/U schedule/R
Corrections	—	C/R own	R through repository	no usable follow-up UI	R/U approve/finalize
Documents	—	R own scope	R assigned-student scope	R assigned-thesis only	A all/soft delete
Notifications	—	R/mark all	R/mark all	R/mark all	R/mark all + delivery log
Reports/archive	—	—	—	—	API-only reports; archive action/list

7.3 Important authorization behavior

Most protected APIs call `withAuth`, which accepts a valid bearer token or Firebase session cookie, looks up an active local user, and authorizes the role from the decoded Firebase claim (`src/lib/firebase/auth.ts:81-108,132-166`; `src/lib/firebase/with-auth.ts:24-56`).

The local `User.role` is not selected or compared. A stale or malicious claim can therefore remain authoritative even if the database role changes. More seriously, the unauthenticated claims endpoint can create that drift deliberately; see RISK-001 in the final section.

Role layouts do not consistently authenticate on the server. APIs protect data, but an anonymous or wrong-role user can render another role's dashboard shell/navigation before API requests fail. Role enforcement belongs in each role layout, not only in selected pages.

8. Dashboard and user-interface inventory

8.1 Public routes

Route	Purpose	Current notes
/	Landing page	Apply Now and Sign In actions.
/apply	Four-step application	Applicant, Research, Documents, Review. Up to ten PDF/ZIP files.
/apply/success	Confirmation	No application reference or tracking link.
/login	Firebase sign-in and server session creation	Role directs dashboard destination.
/logout	Clear server cookies and redirect	Does not explicitly revoke Firebase sessions/tokens.

8.2 Shared dashboard shell

The shared role layout provides a responsive sidebar, active navigation, notifications drawer, sign-out, header, profile menu, loading/error/empty patterns, and a `max-w-7xl` content area. Dashboard summaries refresh every 30 seconds and on focus through SWR.

Current shell problems:

- the profile is hard-coded as `Administrator / admin@pdn.ac.lk` for every role (`src/components/profile-dropdown.tsx:17-35`);
- Profile and Settings link to nonexistent `/dashboard/settings`; the generic `[role]` page renders a bogus workspace instead;
- the real sidebar uses a custom notification trigger that omits the component's unread badge;
- non-admin headings can render “Student Dashboard Dashboard” and equivalent duplicates;
- role layouts do not consistently validate the authenticated user/role server-side.

8.3 Student dashboard and pages

Surface	Current functionality	Accuracy/gap
Overview KPIs	Active Registrations; Proposal Reviews; Ethics Approvals; Overdue Reports; Open Thesis Milestones	“Ethics Approvals” counts packages, not approvals. Overdue derives from obsolete sign-off.
Quick actions	Submit/view progress, proposal status, ethics, thesis, corrections	Useful route launcher; no recent activity or deadlines.

Surface	Current functionality	Accuracy/gap
Proposal	Create/revise, multi-file upload, history/download, status	Multi-file download/history conflict.
Ethics	Submit package, history/download	UI correctly reflects document-only design; downstream labels do not.
Progress	Narrative/files submission and history	Displays obsolete sign-off state; upload can fail after DB commit.
Lifecycle progress	Milestones, percentages, estimated completion, stepper	Treats submissions/files as approvals and can show false completion.
Thesis	Submit/revise/history/download	Multi-file conflict; correction-required status permits alternate resubmission path.
Corrections	Submit/history/download	Student chooses type without enforcing viva result.
Documents	Search/filter/download own scope	Can expose unreleased review attachments.

There is also an unlinked duplicate `/student/progress` route in addition to `/dashboard/student/progress`.

8.4 Supervisor dashboard and pages

Surface	Current functionality	Accuracy/gap
Overview KPIs	Assigned Students; Submitted Proposals; Submitted Reports; Graduated Students	Counts include broader history than labels imply.
Roster	Programme/registration filters, assignment type, expiry, proposal title	Useful operational list.
Student profile	Programme/status/enrolment and workflow cards	“View Proposal History” and “Open Progress Reports” are <code>href="#"</code> .
Proposal page	Informational monitoring notice	Route is named <code>/proposals/evaluate</code> , but no evaluation occurs.
Progress page	View narrative and attached filenames	Component/route still says sign-off though it is view-only.
Documents	Assigned-student repository	Includes thesis documents despite specialized thesis access denying supervisors.

The “Student Roster” quick action links to supervisor home rather than the roster.

8.5 Examiner dashboard and pages

Surface	Current functionality	Accuracy/gap
Overview KPIs	Assigned Theses; Scheduled Vivas; Pending Corrections; Active Examinations	Counts include historical/completed data and correction files rather than cases.
Vivas	Candidate, abstract, current thesis download, venue/date, outcome buttons	Time is omitted in the card; outcome governance is insufficient.
Documents	Thesis-only repository for assigned theses	No proposal/progress evidence access.
Quick actions	Review theses/vivas; Track Corrections	Track Corrections returns home; no correction screen.

There is no UI for proposal evaluation, progress-review assignment work, progress review submission, thesis review report, review attachments, or released-feedback handling, even though some backend services exist.

8.6 Administrator dashboard and pages

Surface	Current functionality	Accuracy/gap
Overview KPIs	Active Staff; Pending Applications; Archived Theses; Overdue Reports; Ethics Documents; Students Under Review; Failed Notifications	Several definitions are misleading; no trends or drill-down tables.
Users	Create, filter, deactivate	No edit/reactivate/reset/search/pagination/self-deactivation guard.
Applications	List submitted, detail/download, admit/reject	Under-review records disappear from list; rejection has no reason.
Proposals	Approve/request revision with comment	Does not display source evidence or examiner evaluations despite instructing admin to review them.
Ethics	List/view/download package	No decision, intentionally.
Supervisor assignment	Create/remove/promote primary/co-supervisor	Good basic controls; concurrency/history/notifications incomplete.
Examiner assignment	Assign examiner and begin examination	No remove/reassign/count policy; one emailed file link.
Viva scheduling	Schedule/reschedule	No review/examiner-count prerequisites.
Thesis finalization	Approve correction; archive and graduate	Correction files are listed without open/download action; decisions lack notes/confirmation.

Surface	Current functionality	Accuracy/gap
Documents	Global repository and soft delete	No retention workflow.
Notification log	Filter, paginate, current-page CSV	No retry/resend; CSV is only the loaded page.

Backend-only admin surfaces: student report, graduation report, thesis pipeline, overdue progress, students under review, archived-record browser, progress review assignment/release, thesis review release, and registration renewal.

8.7 Cross-cutting UI/accessibility findings

Strengths include responsive grids, table overflow, disabled mutation controls, Radix dialog/select/sheet primitives, and widespread loading/error/empty states.

Current accessibility and UX risks include:

- animated loaders without accessible status text or reduced-motion handling;
- continuous landing animation without `prefers-reduced-motion` support;
- `min-width: 360px`, which can fail 320px/400% zoom scenarios;
- no skip link and weak landmark structure, including nested `<main>` elements;
- missing label/control associations in several forms and selects;
- async feedback without consistent live regions or focus management;
- an icon-only remove-supervisor button without an accessible label;
- hidden mobile-sidebar close behavior and uncertain close-on-navigation;
- inconsistent native `alert()/confirm()` versus accessible dialogs;
- no authenticated browser verification of keyboard order, contrast, zoom, or mobile reflow.

9. End-to-end workflow reference

This section describes the current executable workflow, including preconditions, persistence, notifications, and known divergence. “Current result” describes what the audited code does, not what a future policy should require.

9.1 Workflow A — public application and admission

Actors: Applicant, Administrator

Entry: /apply

Primary records: Application, application Document, then User, Student, Registration

Current sequence

15. The applicant enters name, email, telephone, programme, proposed supervisor text, research area, and statement of purpose.
16. The applicant selects one to ten PDF/ZIP files, each with a declared maximum of 10 MB.
17. The browser obtains a 15-minute Supabase signed upload URL through a public API and uploads each file before the application record exists.
18. The browser submits metadata including storage paths. The server validates Zod shape, claimed MIME/size, protected root, and duplicate active application by email.
19. The server creates `Application(SUBMITTED)` and file metadata, then emails every active administrator.
20. Administrator opens the application, downloads evidence, and chooses a permitted status.
21. If `ADMITTED`, the server creates a Firebase user, assigns the `STUDENT` custom claim, creates local `User` and `Student(ACTIVE)`, creates a one-year `Registration(ACTIVE)`, links the application, and sends a welcome email.
22. If `UNDER_REVIEW` or `REJECTED`, only the database status changes.

Evidence: `src/lib/applications/submission.ts:84-135,176-321,324-480`;
`src/app/api/applications/[id]/status/route.ts:24-59`.

Communications

- Submission: active administrators receive email, logged under `APPLICATION_STATUS_CHANGED`; no in-app notification.
- Admission: new student receives welcome email containing temporary password; no explicit in-app item is needed before first login.
- Under review/rejected: no applicant communication.

Current completion and flaws

- **Implemented:** public form, evidence upload, duplicate check, admin review/download, status transition, account/profile/registration provisioning.
- **Not implemented:** applicant reference/status page, save/resume, review ownership, rejection reason, public-recipient notification history.
- Public draft upload/delete is not rate-limited and remains possible using a retained draft identity. Upload uses `upsert`; there is no committed-application lock or object-existence verification.
- Admission cannot reuse/link an existing local user with the applicant email.
- Temporary password generation uses `Math.random()` and sends a reusable password in plaintext (`src/lib/applications/submission.ts:56-65,395-460`; `src/lib/email.ts:240-263`).

9.2 Workflow B — user sign-in, session, and role authorization

Actors: All authenticated roles

Primary records/services: Firebase Auth, Firebase Admin, User, secure cookies

Current sequence

23. Browser signs in through Firebase and receives an ID token.
24. `POST /api/auth/session` verifies the token, looks up the linked active local user, creates a Firebase session cookie, and sets an activity cookie.
25. Cookies are `HttpOnly`, `Secure`, `SameSite=Lax`, path `/`; absolute lifetime is seven days and inactivity timeout is 30 minutes (`src/lib/firebase/admin.ts:114-151`; `src/lib/security/session.ts:1-10`).
26. Protected APIs accept a bearer token or the session cookie, verify Firebase, require an active local user, and authorize the token claim role.
27. The client activity tracker periodically refreshes the activity cookie.
28. Logout clears both application cookies and redirects home.

Current flaws

- Local database role and Firebase claim are not compared on each request (`src/lib/firebase/auth.ts:88-108`).
- `POST /api/auth/claims` has no authentication or authorization and can set arbitrary supported role claims/local roles.
- Role layouts do not consistently block anonymous/wrong-role shell rendering.
- Logout clears local cookies but does not explicitly revoke all Firebase refresh/session tokens; stolen sessions remain bounded mainly by verification/revocation state and expiry.
- Cookie-authenticated mutation routes do not implement visible CSRF tokens or strict Origin/Referer validation. `SameSite=Lax` is useful but not a complete control.

9.3 Workflow C — registration maintenance and renewal

Actors: Student, Administrator, scheduled maintenance

Primary record: Registration

Current sequence

29. Admission creates an active period ending one year after admission.
30. A student owner or Administrator may call the renewal API for any registration.
31. The service archives the current active period, if present, and creates a new one-year active period.
32. If the current expiry is in the future, the new period starts from that expiry; otherwise it starts now.
33. Maintenance marks expired active rows LAPSED.
34. On exactly 14 days before expiry, maintenance emails and creates an in-app reminder.

Evidence: `src/lib/registrations.ts:66-145,147-229`;
`src/app/api/registrations/[id]/renew/route.ts:11-42`.

Current flaws

- No UI exposes renewal.
- Renewal is immediate, with no request, evidence, fee/payment, approval, or reason.
- Repeated early renewal can extend indefinitely; concurrent renewals can create multiple active rows because there is no database invariant/idempotency key.
- Finalization and record archive do not close registrations; graduated/archived students can be reminded and can renew.
- `GET /api/cron/check-registrations` changes state. If `CRON_SECRET` is absent, it deliberately authorizes every request (`src/app/api/cron/check-registrations/route.ts:6-24`).
- No scheduler is committed. Exact-day reminders can be missed or duplicated and have no delivery deduplication.

9.4 Workflow D — supervisor assignment

Actors: Administrator, Supervisor, Student

Primary record: SupervisorAssignment

Current sequence and rules

35. Administrator selects a student and active supervisor.
36. The first assignment is forced to Primary.
37. Application code enforces maximum three supervisors, maximum two co-supervisors, and one primary.

38. Promoting an assignment demotes the current primary transactionally.
39. A person already serving as examiner for the student cannot be assigned as supervisor.
40. New supervisor receives an email naming student, role, and assigning administrator.
41. Administrator can delete an assignment; the row is permanently removed.

Evidence: `src/lib/assignments/supervisors.ts:83-208,215-427`.

Current flaws

- Limits are not database constraints and can be violated by concurrent requests.
- Student receives no assignment/change notice; supervisor receives no promotion/removal notice.
- Assignment email is logged as `APPLICATION_STATUS_CHANGED`.
- Deleted assignments lose history; no start/end dates or reason.
- Archived/graduated student state is not checked.

9.5 Workflow E — proposal submission, revision, examination, and decision

Actors: Student, assigned Supervisor (monitor only), Examiner (backend review), Administrator

Primary records: `ResearchProposal`, proposal Document, `EvaluationForm`

Student submission

42. Student must have an admitted linked application and an unexpired active registration.
43. Browser uploads one to ten PDF/ZIP documents through signed URLs.
44. First proposal is `UNDER_REVIEW` when any supervisor is assigned, otherwise `SUBMITTED`.
45. A rejected proposal can be revised: metadata is updated, integer version increments, prior documents are marked non-current, and the proposal returns to the initial state.
46. Approved proposal is terminal.

Evidence: `src/lib/proposals/submission.ts:113-268,325-480`.

Examiner evaluation backend

47. There is no proposal-examiner assignment model or examiner queue.
48. Any Examiner who learns a proposal ID can submit one evaluation while the proposal is submitted/under review, unless they are also the student's supervisor.
49. Any non-conflicted Examiner can retrieve the evaluations collection, including other examiner feedback/admin fields.
50. Examiner proposal version access is explicitly denied by the proposal service, and the general repository only permits thesis documents for Examiners.
51. Admin UI does not load the evaluation or proposal evidence.

Evidence: `src/lib/proposals/evaluations.ts:186-243,312-455`; denial at `src/lib/proposals/versions.ts:126-159`.

Administrator decision and notifications

Administrator can approve or reject/request revision directly; no evaluation is required. A status change sends email and in-app notification to the student. Optional feedback is included in the email but is not persisted as a decision record.

Multi-file failure

Every file within the new logical version is marked `isCurrentVersion=true` (`src/lib/proposals/submission.ts:375-384,417-446`). The versions service throws unless exactly one document across the proposal is current (`src/lib/proposals/versions.ts:111-123`). Thus a valid multi-file package immediately breaks history/download.

9.6 Workflow F — ethics evidence

Actors: Student, Administrator

Primary records: `EthicsApproval`, `ethics Document`

Current sequence

52. Student must have an approved proposal and active registration.
53. Student submits one non-archived ethics package with one to ten PDF/ZIP files.
54. Active administrators receive email and in-app notification.
55. Administrators can list and download the evidence.
56. There is no decision. `PATCH /api/admin/ethics/[id]/decision` always returns `410 Gone`.

The current migration deliberately removed ethics status, reviewer, review notes, and review timestamps. Evidence: `src/lib/ethics/approvals.ts:152-255,293-455`; migration `prisma/migrations/20260709090000_lifecycle_examiner_reviews_and_multi_uploads/migration.sql:26-34`.

Consequences

- Any ethics package is treated as clearance in thesis preconditions and student progress.
- No approval, rejection, revision, exemption, expiry, committee, or certificate can be represented.
- The name `EthicsApproval` and KPI “Ethics Approvals” overstate the implemented behavior.

9.7 Workflow G — progress reporting and review

Actors: Student, assigned Supervisors, Administrator, Examiner

Primary records: `ProgressReport`, `progress Document`, `ProgressReportReview`

Student submission and supervisor monitoring

57. Student needs an unexpired active registration; academic/archive state is not checked.

58. Student enters a free-text period label and narrative, with zero to ten optional files.
59. Server creates the report and document metadata transactionally, then returns signed upload URLs.
60. Browser uploads the files after the API response.
61. Every active assigned supervisor receives background email and an in-app notification.
62. Supervisors can view assigned students' reports, narratives, and filenames only.

Evidence: `src/lib/progress-reports/submission.ts:76-140,172-309`.

Retired sign-off and panel design

- PATCH `/api/progress-reports/[id]/sign-off` and POST `/api/supervisor/progress-reports/[id]/sign` return 410.
- Review-panel create/read/evaluate routes return 410.
- Legacy sign-off fields, review-panel models, names, badges, and tests remain.

Evidence: `src/lib/progress-reports/sign-off.ts:3-23`; `src/lib/review-panels/index.ts:16-45`.

Examiner-review replacement

Administrator assigns Examiner
 → assigned Examiner submits one immutable text review + optional attachments
 → Administrator adds comments and releases/unreleases the review

Supervisor conflict is checked. The backend does not provide a usable examiner list/queue UI, assignment notification, submission notification, admin release screen, or student/supervisor released-review view.

Review attachments are created with the student ID (`src/lib/progress-reports/reviews.ts:343-357`). The general repository grants a Student any document with their ID (`src/lib/documents.ts:94-154`), without checking `releasedAt`. A student can therefore access an unreleased examiner attachment.

Overdue defect

Maintenance marks an existing report overdue after 30 days while `isSupervisorSignedOff=false`. Because sign-off is retired and no active flow sets the flag, every submitted report eventually becomes overdue. The logic does not detect a missing expected report; it only ages submitted reports.

9.8 Workflow H — thesis submission and document versioning

Actors: Student, Administrator, assigned Supervisors (email/repository), Examiners

Primary records: Thesis, thesis Document

Preconditions and sequence

63. Student must be academically active, have an approved proposal, have an unexpired active registration, and have at least one ethics package.
64. Student submits title, abstract, and one to ten PDF/ZIP files.
65. First submission creates `Thesis(SUBMITTED)` and file metadata.
66. When the current thesis is `CORRECTIONS_REQUIRED`, the same thesis can be resubmitted as the next version and reset to `SUBMITTED`, without using or waiting for correction-package approval.
67. The server commits metadata before returning signed upload URLs; the browser performs storage PUTs afterward.
68. Active administrators receive email. Assigned supervisors receive the progress-report email template with a synthetic period label such as `thesis submission: <title>`.

Evidence: `src/lib/theses/submission.ts:107-295,356-407,409-558`.

Multi-file failure

All files in the logical version are current (`src/lib/theses/submission.ts:439-546`). The download/version service requires exactly one current thesis document (`src/lib/theses/versions.ts:106-119`). Valid multi-file submissions therefore break student/examiner/viva current-download and make an integer version ambiguous.

9.9 Workflow I — thesis examiner assignment and report release

Actors: Administrator, Examiner

Primary record: `ThesisExaminerAssignment`

Assignment

69. Administrator may assign an active Examiner while thesis is `SUBMITTED` or `UNDER_EXAMINATION`.
70. Duplicate and supervisor conflict checks are performed.
71. Assignment does not automatically set `UNDER_EXAMINATION`, require a minimum/maximum examiner count, or support removal/reassignment.
72. Examiner receives an email containing a 15-minute signed link to only the first current thesis document.

Evidence: `src/lib/assignments/examiners.ts:194-318`.

Review backend

73. Assigned Examiner can submit one immutable text review with optional attachments, even while thesis remains `SUBMITTED`.
74. Administrator can add comments and set/clear `releasedAt`.
75. The review is not required before viva scheduling or outcome.

- 76. No usable review UI, notification, or student released-review retrieval path exists.
- 77. Review attachments are linked to the student and leak through the repository before release, as described in Workflow G.

Evidence: `src/lib/theses/reviews.ts:127-281`.

9.10 Workflow J — viva scheduling and outcome

Actors: Administrator, assigned Examiner, Student

Primary record: Viva

Scheduling

- 78. Thesis must be `UNDER_EXAMINATION` and the date must be in the future.
- 79. Administrator schedules or reschedules the one viva per thesis.
- 80. Student and every assigned Examiner receive email and in-app notification containing thesis, date/time, and venue.
- 81. Scheduling does not require an examiner count or submitted/released reviews.

Outcome

- 82. Any assigned Examiner may record the single shared outcome.
- 83. The service immediately maps the outcome to the thesis status.
- 84. There is no scheduled-date check, per-examiner recommendation, quorum/consensus, chair, comments/report, confirmation, recorded-by field, or administrative ratification state.
- 85. The first result changes the thesis status, preventing a second examiner from recording an independent view.
- 86. No outcome notification is sent.

Evidence: `src/lib/vivas.ts:115-149,192-345`.

9.11 Workflow K — corrections, approval, finalization, and graduation

Actors: Student, Administrator

Primary records: `CorrectionDocument`, correction Document, Thesis, Student

Correction package

- 87. Student with own non-archived `CORRECTIONS_REQUIRED` thesis submits one to ten files, a description, and self-selected Minor/Major type.
- 88. Server creates unapproved correction metadata before browser file upload completes.
- 89. Active administrators receive email and in-app notification, using the `CORRECTIONS_REQUIRED` event even though the trigger is “correction submitted.”
- 90. Administrator may approve, setting approved flag/time/actor only.

91. Approval does not change thesis state or notify the student. There is no rejection state, decision reason, or requested revision.

Finalization

92. Administrator may finalize a `CORRECTIONS_REQUIRED` thesis with at least one approved correction, or an already `FINAL_ARCHIVE` thesis.
93. Service sets thesis status to `FINAL_ARCHIVE` and Student academic status to `GRADUATED`.
94. Student receives background archive email and in-app notification.
95. `Thesis.isArchived` and registration state are not updated; user account stays active.

Evidence: `src/lib/theses/corrections.ts:150-423,491-635`.

Current bypasses/inconsistencies

- Student correction type is not checked against viva outcome.
- One approved package is enough even if other packages remain pending.
- Student can instead submit a new thesis version while corrections are required.
- Generic admin thesis status can set states without synchronizing graduation or notifications.
- `FINAL_ARCHIVE` is not closed by the specialized finalization action.

9.12 Workflow L — administrative student-record archive

Actors: Administrator

Primary records: Student plus related soft-archive flags

The archive command sets `Student.isArchived=true`, `AcademicStatus.ARCHIVED`, and archives linked application, proposal, ethics package, and progress reports. Active theses generate warnings but are not changed. Registrations, file metadata, supervisor/examiner assignments, notifications, local user activation, and Firebase access are left intact (`src/lib/admin/archive.ts:37-149`).

There is no restore/unarchive path. Some downstream services block `GRADUATED` but not `ARCHIVED`, so archived records can remain actionable. The archived-record list expects `Thesis.isArchived=true`, but current finalization does not set it.

9.13 Workflow M — document repository and downloads

Actors: All authenticated roles

Primary record: Document

Current behavior

- Administrator: all non-deleted repository documents; can soft delete and can still download deleted items.

- Student: direct `studentId` documents and documents related to their application/proposal/ethics/progress/thesis/corrections.
- Supervisor: equivalent scope for assigned students.
- Examiner: thesis documents for assigned theses only.
- Download returns a Supabase signed URL valid for 15 minutes.
- Search supports text, category, tag, dates, pagination, and status-derived tags.

Evidence: `src/lib/documents.ts:63-266,359-430,816-1034`.

Policy and integrity problems

- `REVIEW_ATTACHMENT` is included in repository types (`src/lib/documents.ts:63-71`) and student/supervisor direct-ID scopes have no release check.
- Specialized thesis service explicitly denies Supervisors, while the general repository grants assigned Supervisors thesis documents.
- Examiner download audit covers only one current-thesis endpoint. Version downloads, repository downloads, and emailed signed links are not logged.
- `Document` has several nullable parent IDs and denormalized `studentId`; no database check enforces one valid parent or consistency.
- MIME and size are based mainly on client-declared metadata; no signature/content sniff, malware scan, checksum, or storage-finalize verification exists.

9.14 Workflow N — notification consumption and delivery audit

Actors: All authenticated roles; Administrator for delivery log

96. Notification sidebar fetches the most recent eight records once on mount.
97. Drawer displays title, message, event, and relative time.
98. User can mark all notifications read; no individual-read operation exists.
99. Notification records have no destination/action URL.
100. Administrator can filter and paginate `NotificationLog` and export the current loaded page to CSV.

The API does not return an authoritative unread count. The custom sidebar trigger omits the default badge, and the UI would otherwise estimate only from the eight fetched records. There is no polling/on-open refresh, “view all”, retry/resend, or failed-delivery repair flow.

10. Notifications and email trigger catalogue

10.1 Delivery architecture

`sendEmail()` sends through Nodemailer and records SENT or FAILED in `NotificationLog`; SMTP errors are converted into a result rather than thrown (`src/lib/email.ts:79-135`). The central `notify()` dispatcher sends email and then writes an in-app `Notification`; the two writes are not transactional (`src/lib/notifications.ts:147-365`). Several workflows bypass the dispatcher and send only email.

“Fire-and-forget” calls use unresolved background promises inside the request process. In serverless/short-lived runtimes, completion is not guaranteed. There is no outbox, job queue, retry schedule, idempotency key, or dead-letter state.

10.2 Master trigger matrix

Trigger	Recipient	Channel actually used	Subject/title and main content	Stored event classification	Current issue
Public application submitted	All active Administrators	Email + email log	New <programme> application submitted; applicant, email, research area	APPLICATION_STATUS_CHANGED	No in-app; event misclassified
Application set under review	Applicant	None	—	—	Silent; applicant is not a User
Application rejected	Applicant	None	—	—	Silent; no rejection reason
Application admitted/account created	New Student	Background email + log	Your student account is ready; role, temporary password, login URL	APPLICATION_STATUS_CHANGED	Plaintext weak temporary password; no durable invite/forced reset
Admin creates staff/student account	Created user	Background email + log	Same welcome template	APPLICATION_STATUS_CHANGED	Same password/invite concern
Supervisor assigned	Supervisor	Email + log	New supervisor assignment: <role>; student, assigner	APPLICATION_STATUS_CHANGED	No student/in-app; wrong event; no promotion/removal message
Proposal status changed	Student	Email + in-app + log	Proposal status updated: <status>; title, optional feedback	PROPOSAL_STATUS_CHANGED	Feedback not persisted as decision record
Proposal Examiner submits review	Active Administrators	Email + in-app + log	Proposal review received: <title>; examiner, student, feedback	Email log PROPOSAL_STATUS_CHANGED; in-app EXAMINER_REVIEW_S	Inconsistent taxonomy; no admin UI

Trigger	Recipient	Channel actually used	Subject/title and main content	Stored event classification	Current issue
				UBMITTED	
Ethics package submitted	Active Administrators	Email + in-app + log	Ethics approval submitted: <title>; student, title	ETHICS_APPROVAL_S UBMITTED	Wording implies decision workflow that does not exist
Progress report submitted	All active assigned Supervisors	Background email + in-app + log	Progress report submitted for <period>; student, period	PROGRESS_REPORT_S UBMITTED	No deep link; notification can precede failed file upload
Progress review assigned	Examiner	None	—	EXAMINER_REVIEW_A SSIGNED defined but unused	Missing
Progress review submitted	Administrator	None	—	EXAMINER_REVIEW_S UBMITTED exists but this flow does not call it	Missing
Progress review released	Student/Supervisor	None	—	ADMIN_REVIEW_RELE ASED defined but unused	Missing and no release UI
Thesis submitted	Active Administrators	Email + log	New thesis submission: <title>; student, programme, title	APPLICATION_STATU S_CHANGED	No in-app; wrong event; can precede failed upload
Thesis submitted	Active assigned Supervisors	Email + log	Progress-report template with synthetic “period”	PROGRESS_REPORT_S UBMITTED	Wrong template/event; no in-app
Thesis Examiner assigned	Examiner	Email + log	New thesis examiner assignment: <title>; student, assigner, 15-minute URL	APPLICATION_STATU S_CHANGED	Wrong event; one first file only; link likely expires before use; no in-app
Thesis review submitted	Administrator	None	—	—	Missing
Thesis review released	Student	None	—	—	Missing and no retrieval path
Viva scheduled/rescheduled	Student and assigned Examiners	Email + in-app + log	Viva scheduled for thesis: <title>; server-formatted date/time and venue	VIVA_SCHEDULED	No timezone label/deep link; Examiner card omits time
Viva outcome recorded	Student/Admin/Supervisors/other Examiners	None	—	—	Critical lifecycle transition is silent
Viva requires corrections	Student	None	—	CORRECTIONS_REQUI RED defined but not used here	Student is not told requirements

Trigger	Recipient	Channel actually used	Subject/title and main content	Stored event classification	Current issue
Student submits correction package	Active Administrators	Email + in-app + log	Correction document submitted: <title>; student, chosen type	CORRECTIONS_REQUESTED	Event describes requirement, but trigger is submission
Correction approved	Student	None	—	—	Silent; thesis state unchanged
Thesis finalized/archived	Student	Background email + in-app + log	Your thesis has been archived: <title>; completion message	THESIS_ARCHIVED	Generic status endpoint sends nothing; background reliability
Registration exactly 14 days from expiry	Student	Email + in-app + log	Registration expiry reminder: 14 days remaining; date, renewal warning	REGISTRATION_EXPIRY_APPROACHING	No dedup; depends on scheduler/exact day
Registration lapses	Student/Admin	None	—	—	Silent
Registration renewed	Student/Admin	None	—	—	Silent
Examiner current-thesis download through one endpoint	Notification log only	Audit row	Thesis download accessed: <id>	THESIS_DOWNLOADED	Other download paths are not audited; not a user notification

10.3 Defined event catalogue and actual use

The schema defines 15 events (`prisma/schema.prisma:76-92`). Five are effectively dead in the current workflow: `ETHICS_APPROVAL_STATUS_CHANGED`, `PROGRESS_REPORT_SIGNED_OFF`, `SUPERVISOR_SUBMISSION_AVAILABLE`, `EXAMINER_REVIEW_ASSIGNED`, and `ADMIN_REVIEW_RELEASED`. `THESIS_DOWNLOADED` is an audit marker, not a notification. Several active emails reuse unrelated events, making KPI/filter/audit interpretation unreliable.

10.4 Template security and content quality

Email HTML templates interpolate names, titles, research areas, feedback, venues, and similar values directly into HTML strings without escaping—for example proposal feedback at `src/lib/email.ts:166-191`. A malicious stored value can alter email markup or create deceptive links/content. Use a shared HTML-escape helper or a templating system with escaping enabled.

The viva date uses `server.toLocaleString()` without an explicit `timezone/locale` (`src/lib/email.ts:614-639`). Store UTC, render the configured institutional timezone, and include the timezone abbreviation in email and UI.

11. Administrator features, reports, and maintenance

11.1 Currently exposed administrator functions

Function	Current behavior	Principal limitation
User management	Create role account and profile; filter; deactivate	No edit/reactivate/reset/search/pagination; unsafe password invite; no last-admin/self guard visible
Application management	List submitted, inspect, download, change status, admit/reject	Under-review queue missing; no owner, notes, reason, or history
Proposal decisions	List and set approved/rejected with optional feedback	Decision screen lacks proposal evidence and examiner reviews
Ethics evidence	List and download	No decision by current design
Supervisor assignment	List/create/remove/promote	No history/date/reason; race-prone limits
Examiner assignment	Assign active Examiner, email first file	No remove/reassign/count policy, review queue, or in-app item
Viva	Schedule/reschedule	No prerequisites beyond thesis status/future date
Corrections/finalization	Approve correction, archive/graduate	No download in decision screen, rejection/notes/confirmation, or atomic registration closure
Document administration	Search and soft delete	No retention schedule, restore, legal-hold, or parent-integrity view
Notification delivery audit	Filter/paginate/export current page	No retry/resend, full filtered export, delivery latency, or alerting

11.2 Backend reports not exposed in the dashboard

API	Information available	Current UI status
/api/admin/reports/students	Student/programme/status/registration overview and CSV-capable response	Backend-only
/api/admin/reports/graduations	Graduated student/thesis information	Backend-only
/api/admin/reports/thesis-pipeline	Thesis states and pipeline data	Backend-only
/api/admin/reports/overdue-progress	Reports marked overdue	Backend-only; underlying overdue definition is flawed

API	Information available	Current UI status
/api/admin/students/under-review	Student under-review list	KPI exists, no drill-down
/api/admin/archived	Archived records	No browser/restore UI; thesis archive flag inconsistency limits results

Recommended dashboard work should begin by correcting metric definitions, then add evidence-backed queues and drill-downs rather than decorative charts.

11.3 Maintenance jobs

The single maintenance endpoint performs both registration maintenance and overdue progress marking. It is not connected to a committed scheduler. The endpoint should be split into idempotent jobs, use mandatory secret/signature authentication, accept only POST, record run/lease state, deduplicate reminders, expose health metrics, and define retry behavior.

12. API route catalogue

The following catalogue records the actual route-handler surface at the baseline. Service-level ownership/conflict checks still apply after the role wrapper. “Retired” means the route exists but the service returns 410 Gone.

12.1 Public, authentication, dashboard, and maintenance APIs

Route	Method	Access	Purpose/status
/api/applications	POST	Public	Submit public application
/api/applications/upload-url	POST	Public	Create signed draft upload URL
/api/applications/upload	POST, DELETE	Public	Server upload or delete draft evidence
/api/auth/session	POST, PATCH, DELETE	Public/session	Create, refresh activity, or clear session cookies
/api/auth/me	GET	All authenticated roles	Current authenticated context
/api/auth/claims	POST	Public (defect)	Sets Firebase claim and local role; critical vulnerability
/api/dashboard/[role]/summary	GET	All authenticated roles; requested role checked	Role KPI/action summary
/api/notifications	GET, PATCH	All authenticated roles	Recent notifications; mark all read
/api/cron/check-registrations	GET	Secret only if configured; otherwise public	Registration reminders/lapse and overdue marking
/api/test/rbac	GET	Administrator	Test endpoint present in application route tree; remove/disable for production

12.2 Application and user administration APIs

Route	Method	Access	Purpose/status
/api/applications	GET	Administrator	List applications, status filter supported
/api/applications/[id]	GET	Administrator	Application detail
/api/applications/[id]/status	PATCH	Administrator	State transition/admission provisioning
/api/applications/[id]/documents/[docId]/download	GET	Administrator	Signed evidence download

Route	Method	Access	Purpose/status
/api/admin/users	GET, POST	Administrator	List/create role accounts
/api/admin/users/[id]/deactivate	PATCH	Administrator	Deactivate user and linked Firebase account behavior
/api/students/[id]	GET, PATCH	Student/Supervisor/Administrator or with service scope	Student profile view/update
/api/students/[id]/progress	GET	Student owner	Lifecycle progress projection
/api/students/[id]/archive	PATCH	Administrator	Administrative record archive
/api/registrations/[id]/renew	POST	Student owner or Administrator	Immediate registration renewal

12.3 Assignment, proposal, and ethics APIs

Route	Method	Access	Purpose/status
/api/assignments/supervisors	GET, POST	Administrator	List/create assignment
/api/assignments/supervisors/[id]	PATCH, DELETE	Administrator	Promote/update or remove assignment
/api/proposals	GET, POST	Student	Own proposal view/submission/revision
/api/proposals/upload-url	POST	Student	Proposal signed upload URL
/api/proposals/[id]/status	PATCH	Administrator	Proposal decision
/api/proposals/[id]/versions	GET	Student/Supervisor/Administrator; Examiner wrapper admitted but service denies	Version list; fails for valid multi-file current version
/api/proposals/[id]/versions/[v]/download	GET	Same service scope	Signed version download
/api/proposals/[id]/versions/[v]	—	None	Empty/inert route file; remove or implement
/api/proposals/[id]/evaluations	GET	Examiner/Administrator	Evaluation list; Examiner access is not assignment-based
/api/proposals/[id]/evaluations	POST	Examiner	Submit one evaluation without assignment
/api/ethics	GET, POST	Student	Own ethics package view/submission
/api/ethics/upload-url	POST	Student	Ethics signed upload URL
/api/admin/ethics	GET	Administrator	List packages
/api/admin/ethics/[id]/decis	PATCH	Administrator	Retired; always `410`

Route	Method	Access	Purpose/status
ion			

12.4 Progress-report APIs

Route	Method	Access	Purpose/status
/api/student/progress-reports	GET, POST	Student	Own report history/submission
/api/supervisor/progress-reports	GET	Supervisor	Assigned-student monitoring
/api/progress-reports/[id]/sign-off	PATCH	Supervisor	Retired; always `410`
/api/supervisor/progress-reports/[id]/sign	POST	Supervisor/Administrator wrapper	Retired; always `410`
/api/review-panels	POST	Administrator	Retired; always `410`
/api/review-panels/[id]	GET	Administrator/Supervisor	Retired; always `410`
/api/progress-reports/[id]/panel-evaluation	POST	Supervisor	Retired; always `410`
/api/progress-reports/[id]/reviews	POST	Administrator	Assign Examiner review; backend-only
/api/progress-report-reviews/[id]	POST	Assigned Examiner	Submit review; backend-only
/api/progress-report-reviews/[id]	PATCH	Administrator	Comment/release review; backend-only

12.5 Thesis, viva, correction, and document APIs

Route	Method	Access	Purpose/status
/api/theses	POST	Student	Submit/revise thesis
/api/theses/[id]/status	PATCH	Administrator	Generic thesis transition
/api/theses/[id]/versions	GET	Student/Examiner/Administrator; Supervisor wrapper admitted but service denies	Version list; multi-file defect
/api/theses/[id]/versions/[v]/download	GET	Same service scope	Version download
/api/theses/[id]/download	GET	Assigned Examiner	Current download and limited audit
/api/assignments/examiners	POST	Administrator	Assign thesis Examiner
/api/thesis-examiner-	POST	Assigned Examiner	Submit thesis review; backend-

Route	Method	Access	Purpose/status
assignments/[id]/review			only
/api/thesis-examiner-assignments/[id]/review	PATCH	Administrator	Comment/release review; backend-only
/api/vivas	POST	Administrator	Schedule/reschedule viva
/api/vivas/[id]	GET	Administrator/assigned Examiner	Viva detail
/api/vivas/[id]/outcome	POST	Assigned Examiner	Record shared outcome
/api/theses/[id]/corrections	GET	Administrator	List correction packages
/api/theses/[id]/corrections	POST	Student owner	Submit correction package
/api/theses/[id]/corrections/[cid]/approve	PATCH	Administrator	Approve package
/api/theses/[id]/archive	PATCH	Administrator	Finalize/archive and graduate
/api/documents	GET	All authenticated roles with service scope	Search repository
/api/documents/[id]	GET	All authenticated roles with service scope	Signed download
/api/documents/[id]	PATCH	Administrator	Soft delete

12.6 Administrator report/audit APIs

Route	Method	Access	Purpose/status
/api/admin/proposals	GET	Administrator	Proposal decision list
/api/admin/notification-log	GET	Administrator	Email/audit delivery log
/api/admin/reports/students	GET	Administrator	Student report; backend-only
/api/admin/reports/graduations	GET	Administrator	Graduation report; backend-only
/api/admin/reports/thesis-pipeline	GET	Administrator	Thesis pipeline report; backend-only
/api/admin/reports/overdue-progress	GET	Administrator	Overdue report; backend-only
/api/admin/students/under-review	GET	Administrator	Under-review list; no UI drill-down
/api/admin/archived	GET	Administrator	Archived-record list; no UI

13. Testing, quality, security, and operational readiness

13.1 What the current tests establish

- Core validation, state-map helpers, role wrappers, notification dispatch, service branches, and UI components have meaningful automated coverage.
- The full Vitest suite passed at the baseline: 79 files and 243 tests.
- The production compiler/build passed, so imports, types used by the build, and page generation are internally consistent at the audited snapshot.
- Retired routes have tests that intentionally expect 410, confirming retirement is deliberate rather than accidental.

13.2 What the tests do not establish

- Most integration-labelled tests mock Prisma, Supabase, Firebase, SMTP, and signed URLs.
- No test proves a real browser can complete the entire lifecycle against disposable external services.
- No standard script runs the four Playwright specs; no `playwright.config.*` was found.
- No test covers concurrent supervisor assignment/registration renewal invariants with a real database.
- No end-to-end test verifies multi-file version downloads, upload-finalize failures, review confidentiality before/after release, repeated viva cycles, cron deduplication, or graduation/registration consistency.
- No coverage threshold, mutation testing, migration smoke test, accessibility automation, performance/load test, or dependency gate is configured.

13.3 Security posture summary

Positive controls include runtime Zod validation in many routes/services, Prisma query APIs rather than string SQL, sanitized storage filenames/roots, protected object-store roots, short-lived signed download URLs, secure/HttpOnly/SameSite cookies, active-user checks, role/resource scoping, no raw React HTML sinks found, and ignored environment files.

Material weaknesses include the public claims endpoint, claim/database drift, review attachment leakage, missing CSRF/origin defenses, fail-open cron auth, public upload abuse surface, client-declared file metadata without content inspection, unescaped email HTML, missing application security headers, public source-map risk, incomplete session invalidation, and vulnerable dependencies. Detailed evidence and remedies are in the final risk register.

13.4 Dependency audit snapshot

`npm audit --omit=dev --json` on 18 July 2026 reported:

Severity	Count
Critical	1
High	11
Moderate	26
Low	2
Total	40

Direct affected packages included `next`, `nodemailer`, `firebase`, `firebase-admin`, `@sentry/nextjs`, `postcss`, and the installed Playwright package. The count includes transitive dependencies and does not prove each advisory is reachable through PGLMS, but it requires a controlled upgrade and reachability/regression review before production use.

13.5 Production-readiness checklist

Control	Baseline state
Production build	Passed locally
CI build/test gate	Missing
Deployment manifest	Missing
Environment template/validation	Missing/partial
Database migration pipeline	Missing
Live integration/E2E	Missing
Scheduler definition/health	Missing
Email retry/outbox	Missing
Security headers	Not visible; verify edge
Rate limiting/WAF	Not visible; verify edge
Malware/content scanning	Missing
Audit trail for domain decisions	Missing
Backup/restore evidence	Not visible
Retention/privacy policy	Not visible

Control	Baseline state
Monitoring/alerts	Sentry code present; operational verification required
Accessibility browser audit	Not performed

14. Source map for future developers

Concern	Primary files/directories
Schema/state	prisma/schema.prisma, prisma/migrations, src/lib/prisma/*-status.ts
Authentication/RBAC	src/lib/firebase/admin.ts, auth.ts, with-auth.ts, authorization.ts; src/app/api/auth
Applications/admission	src/lib/applications, src/app/api/applications, src/components/application, admin application components
Registration	src/lib/registrations.ts, renewal route, cron route
Supervisor/examiner assignments	src/lib/assignments, assignment routes/components
Proposal	src/lib/proposals, proposal routes/components
Ethics	src/lib/ethics, ethics routes/components
Progress reports/reviews	src/lib/progress-reports, src/lib/review-panels, corresponding routes/components
Thesis/reviews/corrections	src/lib/theses, thesis routes/components
Viva	src/lib/vivas.ts, src/app/api/vivas, viva components
Documents/storage	src/lib/documents.ts, src/lib/storage.ts, document routes/repository component
Notification/email	src/lib/notifications.ts, src/lib/email.ts, notification routes/components/admin log
Dashboards	src/lib/dashboard/summary.ts, src/components/dashboard, role page trees
Reports/archive	src/lib/admin/system-reports.ts, archive.ts, report routes
Tests	tests/unit, tests/integration, tests/e2e

15. Living progress register

15.1 Change log

Version	Date	Baseline/change	Author/owner	Verification
1.0	18 Jul 2026	Initial full repository audit and implementation baseline	Codex audit; project owner to confirm	Full Vitest and production build passed; source audits completed

15.2 Initial improvement register

This register is the working index. Keep detailed acceptance evidence in the linked issue/PR and update the corresponding capability/risk rows when verified.

Change ID	Related risks/capabilities	Proposed outcome	Priority	Status	Owner	Acceptance evidence
CHG-001	RISK-001, CAP-003	Remove/protect claims endpoint; make role authority consistent	P0	Proposed	Unassigned	Unauthorized claim test returns 401/403; admin-only audited role change; token revocation verified
CHG-002	RISK-002, CAP-007/012/015/021	Enforce release-aware review confidentiality	P0	Proposed	Unassigned	Student/Supervisor receive 403 before release and allowed view after intended release
CHG-003	RISK-003, CAP-006/013	Add logical proposal/thesis version entities supporting multiple files	P0	Proposed	Unassigned	1–10 files per version list/download correctly; one current logical version DB constraint
CHG-004	RISK-004/005, CAP-007/012/015	Build assigned Examiner queues, review UIs, evidence access, and release presentation	P1	Proposed	Unassigned	Complete role E2E for proposal/progress/thesis review with notifications
CHG-005	RISK-006, CAP-008	Decide and implement formal ethics policy or rename/document evidence-only model	P1	Proposed	Unassigned	Approved policy, state model, UI, downstream gate, and tests agree
CHG-006	RISK-007, CAP-009/010/011	Replace retired sign-off/panel semantics and correct overdue reporting	P1	Proposed	Unassigned	No stale routes/labels; missing-report SLA uses defined cycles/due dates
CHG-007	RISK-008, CAP-017	Implement governed per-Examiner viva recommendations and	P1	Proposed	Unassigned	Quorum, reports, chair/admin decision, audit, date guards,

Change ID	Related risks/capabilities	Proposed outcome	Priority	Status	Owner	Acceptance evidence
		ratification				notices tested
CHG-008	RISK-009, CAP-001/006/009/013/018	Introduce staged, verified, idempotent upload finalization	P1	Proposed	Unassigned	Failed PUT cannot create final submission; orphan cleanup and retry tests pass
CHG-009	RISK-010/011, CAP-022/023	Normalize events and add transactional outbox/retry/deep links	P1	Proposed	Unassigned	Required trigger matrix covered; dedup/retry/idempotency verified
CHG-010	RISK-012, CAP-004/019/020	Make renewal/finalization/archive state transitions consistent	P1	Proposed	Unassigned	One active registration constraint; no renewal after graduation/archive; atomic finalization
CHG-011	RISK-013, CAP-024/025	Correct dashboards, dead links, profile, queues, and report pages	P2	Proposed	Unassigned	KPI definitions signed off; every action/drill-down resolves correctly
CHG-012	RISK-014/015/016/017	Security hardening: CSRF, cron, upload abuse, headers, email escape	P1	Proposed	Unassigned	Security regression suite and runtime header scan pass
CHG-013	RISK-018	Upgrade vulnerable dependencies with regression testing	P1	Proposed	Unassigned	Audit exceptions documented; zero unaccepted critical/high reachable findings
CHG-014	RISK-019, CAP-029	Add CI, disposable integration services, E2E, accessibility, migration checks	P1	Proposed	Unassigned	Required branch checks run build/test/E2E/a11y/migrations
CHG-015	RISK-020, CAP-026	Add append-only domain transition/decision/audit history	P2	Proposed	Unassigned	Actor, old/new state, reason, version, timestamp visible and immutable
CHG-016	RISK-021	Consolidate archive/finalization terminology and retention behavior	P2	Proposed	Unassigned	One approved state diagram; archive list/restore/retention tests pass
CHG-017	RISK-022, CAP-030	Establish accessible shared UI patterns and browser verification	P2	Proposed	Unassigned	Keyboard, 320px/400% zoom, contrast, reduced motion, live-region checks pass

15.3 Update procedure

For every accepted change:

101. Record the issue/decision and assign an owner.
102. Update the relevant workflow “Current sequence” only after implementation.
103. Change Capability status only after end-to-end verification.
104. Update the risk status to Mitigated/Accepted/Deferred and cite evidence.
105. Add a change-log row with commit/PR, migration notes, tests, and deployment verification.
106. Regenerate the DOCX from the canonical Markdown and visually verify it.

16. Current flaws, risks, and recommended solutions

This is the final and authoritative risk section for the baseline. Priorities are: **P0** immediate stop-ship; **P1** required before production or the affected lifecycle is relied upon; **P2** important correctness/operability work; **P3** planned quality improvement.

16.1 Priority summary

Priority	Risks	Release interpretation
P0	RISK-001 to RISK-003	Do not expose the current build to untrusted users or rely on multi-file review workflows until corrected.
P1	RISK-004 to RISK-019	Required to make lifecycle decisions reliable, secure, and supportable.
P2	RISK-020 to RISK-031	Required for maintainability, transparency, accessibility, and coherent operations.
P3	RISK-032 to RISK-034	Quality/clarity improvements that should follow core corrections.

16.2 Detailed security findings

RISK-001 — unauthenticated role-claim assignment

- **Rule ID:** NEXT-AUTH-001 / PGLMS-SEC-001
- **Severity/Priority:** Critical / P0
- **Location:** `src/app/api/auth/claims/route.ts:6-30`; `src/lib/firebase/claims.ts:20-31`
- **Evidence:** The public POST parses `userId`, `firebaseUid`, and any supported role, then calls Firebase Admin to set the custom claim and updates the target local user's Firebase UID and role. No authentication, Administrator check, target validation, or anti-self-promotion control is called.
- **Impact:** Any network caller can promote an account to Administrator, relink a local user to an attacker-controlled Firebase identity, or alter another role. This compromises all PGLMS data and actions.
- **Fix:** Remove the route if it is a development helper. Otherwise require an already-authenticated Administrator, verify the target identity, restrict allowed transitions, prevent last-admin/self-dangerous changes, revoke target sessions, write an immutable audit record, and require a recent privileged re-authentication.
- **Mitigation:** Block the route at the edge immediately while the code fix is prepared; inspect Firebase claims and local role/link changes for unauthorized activity.

- **False-positive notes:** None visible in repository. Protection at an external gateway could reduce reachability but does not make the application safe; verify deployed routing urgently.

RISK-002 — unreleased examiner attachments exposed to students/supervisors

- **Rule ID:** NEXT-AUTH-001 / PGLMS-SEC-002
- **Severity/Priority:** High / P0
- **Location:** `src/lib/documents.ts:63-71,94-236,816-834;`
`src/lib/proposals/evaluations.ts:357-372;` `src/lib/progress-reports/reviews.ts:343-357;` `src/lib/theses/reviews.ts:193-207`
- **Evidence:** Review documents are `REVIEW_ATTACHMENT` rows with the candidate's `studentId`. The repository includes that document type and grants Student/Supervisor access based on direct student ID or assigned-student scope. It never checks the parent review's `releasedAt`.
- **Impact:** Confidential examiner feedback/evidence may be downloaded before administrative release, undermining examination integrity and privacy.
- **Fix:** Introduce explicit review-document parent relations and release-aware authorization. Do not grant access merely from `studentId`; expose attachments only through a review endpoint that checks actor, assignment, and release. Backfill and test existing rows.
- **Mitigation:** Temporarily exclude `REVIEW_ATTACHMENT` from Student/Supervisor repository queries and direct downloads.
- **False-positive notes:** Text feedback may remain hidden elsewhere, but the attachment metadata/download path is demonstrably release-blind.

RISK-003 — valid multi-file submissions break current/version downloads

- **Rule ID:** PGLMS-INTEGRITY-001
- **Severity/Priority:** High / P0
- **Location:** proposal create `src/lib/proposals/submission.ts:375-384,417-446` versus invariant `src/lib/proposals/versions.ts:111-123`; thesis create `src/lib/theses/submission.ts:439-546` versus invariant `src/lib/theses/versions.ts:106-119`
- **Evidence:** One logical submission accepts up to ten documents and marks every file current. Read services throw 409 unless exactly one current document exists.
- **Impact:** A valid user action makes history and current-download APIs unusable, blocks Examiner/viva evidence access, and makes version selection ambiguous.
- **Fix:** Create `ProposalVersion` and `ThesisVersion` aggregates with one current version per parent and many files per version. Make reviews/assignments point to the logical version. Add a DB uniqueness mechanism and migration for existing rows.
- **Mitigation:** Until redesigned, limit each proposal/thesis version to one file and communicate that restriction; do not silently accept multiple files.

- **False-positive notes:** None; the conflicting invariants are explicit and covered by separate paths.

RISK-004 — Firebase claim/database role drift

- **Rule ID:** NEXT-AUTH-001 / PGLMS-SEC-003
- **Severity/Priority:** High / P1
- **Location:** `src/lib/firebase/auth.ts:81-108,132-166`
- **Evidence:** Local lookup selects `id`, `firebaseUid`, `email`, and `isActive`, but not `role`; returned authorization role is solely `decodedToken.role`.
- **Impact:** A stale role claim may preserve privileges after a local role change. Combined with claims synchronization errors, the database and authorization layer can disagree.
- **Fix:** Make one authority explicit—prefer local role for every request or require exact equality and fail closed. Revoke/refresh sessions on role/deactivation changes and audit mismatches.
- **Mitigation:** Run a reconciliation job comparing Firebase claims with local roles and investigate discrepancies.
- **False-positive notes:** Firebase claim revocation may be performed operationally, but equality is still not enforced by this code.

RISK-005 — cookie-authenticated mutations lack explicit CSRF/origin validation

- **Rule ID:** NEXT-CSRF-001 / REACT-CSRF-001
- **Severity/Priority:** High / P1
- **Location:** state-changing Route Handlers across `src/app/api`; session cookie options at `src/lib/firebase/admin.ts:136-151`; no CSRF/Origin/Referer checks found by repository scan
- **Evidence:** UI uses cookie-authenticated requests and the server accepts session cookies. Mutation routes rely on `SameSite=Lax` but do not validate a CSRF token or strict request Origin/Referer.
- **Impact:** Cross-site request scenarios, browser/platform changes, subdomain compromise, or unsafe GET behavior can trigger authenticated actions.
- **Fix:** Add a centralized CSRF strategy for cookie-authenticated POST/PUT/PATCH/DELETE APIs, strict Origin/Host validation for JSON endpoints, and tests. Keep `SameSite` as defense in depth.
- **Mitigation:** Require a non-simple custom header and strict same-origin checks at the edge while introducing tokens.
- **False-positive notes:** An external gateway could add checks; no evidence is present. Bearer-only calls are not CSRF-prone, but these handlers also accept cookies.

RISK-006 — fail-open, state-changing cron GET

- **Rule ID:** NEXT-CSRF-001 / NEXT-AUTH-001 / PGLMS-SEC-004
- **Severity/Priority:** High / P1
- **Location:** `src/app/api/cron/check-registrations/route.ts:6-24`

- **Evidence:** Missing `CRON_SECRET` returns authorized, and GET performs database updates and sends notifications.
- **Impact:** An unauthenticated caller can repeatedly lapse records, mark reports overdue, and generate duplicate email/in-app reminders; caches/crawlers may also invoke GET.
- **Fix:** Fail closed if secret/signature configuration is absent, use POST, validate timestamped HMAC or platform job identity, add idempotent run/delivery keys, and rate limit.
- **Mitigation:** Block the route publicly and set a strong secret immediately.
- **False-positive notes:** A private network route could reduce exposure; repository/deployment evidence is absent and fail-open behavior remains unsafe.

RISK-007 — public upload abuse and incomplete file validation

- **Rule ID:** NEXT-FILES-001 / NEXT-DOS-001 / REACT-FILE-001
- **Severity/Priority:** High / P1
- **Location:** public application upload routes; constraints at `src/lib/storage.ts:197-249`; upload upsert at `src/lib/storage.ts:303-317`
- **Evidence:** Public callers can request upload capacity without authentication/rate limiting/CAPTCHA. Validation uses claimed MIME and size; no content signature, malware/archive scan, checksum, quarantine, or object-finalize verification is present.
- **Impact:** Storage/SMTP/DB abuse, malicious or mislabeled files, ZIP bombs, orphaned objects, and post-submission overwrite/delete using retained draft identity.
- **Fix:** Add edge/app rate limits and abuse protection; issue server-bound draft tokens; quarantine uploads; validate magic bytes, actual object size/checksum, archive limits, and malware status; lock finalized paths; expire/clean drafts.
- **Mitigation:** Temporarily lower quotas, disable server upload fallback if unnecessary, restrict bucket policy, monitor anomalies, and disallow ZIP until scanning exists.
- **False-positive notes:** Supabase/edge quotas may exist but were not visible; content verification is absent in application code.

RISK-008 — unescaped user-controlled values in HTML email

- **Rule ID:** PGLMS-SEC-005 (output encoding)
- **Severity/Priority:** Medium / P1
- **Location:** email template builders, e.g. `src/lib/email.ts:144-214,266-317,614-639`
- **Evidence:** Names, titles, feedback, research area, and venue are interpolated directly into HTML strings.
- **Impact:** Stored values can inject deceptive markup/links or malformed content into trusted institutional emails; some clients may expose additional HTML risks.
- **Fix:** Escape every text value by default or use a templating library with contextual auto-escaping; validate URLs separately; add hostile-input tests.

- **Mitigation:** Strip/control HTML-significant characters from high-risk fields until escaping is centralized.
- **False-positive notes:** Email clients often sanitize scripts, but markup/phishing injection remains possible and should not rely on client behavior.

RISK-009 — missing application security headers and possible public source maps

- **Rule ID:** NEXT-HEADERS-001 / NEXT-CSP-001 / REACT-HEADERS-001
- **Severity/Priority:** Medium / P1
- **Location:** `next.config.mjs:4-6`; no header/middleware configuration found
- **Evidence:** No CSP, X-Content-Type-Options, clickjacking defense, Referrer-Policy, or Permissions-Policy is configured in the repository. Production browser source maps are enabled.
- **Impact:** Reduced browser defense-in-depth; possible code/internal-path disclosure and easier exploit development if maps are public.
- **Fix:** Add a tested CSP and baseline headers centrally, verify at runtime, and configure Sentry source-map upload/removal so maps are not publicly served unless explicitly accepted.
- **Mitigation:** Set headers and source-map blocking at the CDN/edge immediately.
- **False-positive notes:** Headers/source-map removal may be supplied by hosting/Sentry; verify actual deployed responses and artifacts before closing.

RISK-010 — vulnerable dependency graph

- **Rule ID:** NEXT-SUPPLY-001 / REACT-SUPPLY-001
- **Severity/Priority:** High / P1
- **Location:** `package.json:16-61`, `package-lock.json`; audit snapshot 18 July 2026
- **Evidence:** Production dependency audit reports 40 known findings, including one critical and eleven high. Direct affected packages include framework/auth/email/monitoring dependencies.
- **Impact:** Reachable advisories may permit denial of service, request/header issues, path/URL confusion, or other compromise depending on code path.
- **Fix:** Create an upgrade branch, update Next.js and direct packages to supported patched versions, regenerate lockfile, run full regression/migration/E2E tests, review breaking changes, and configure continuous dependency alerts/audit gates.
- **Mitigation:** Assess advisory reachability, disable unused vulnerable features, apply edge controls, and document time-bounded exceptions.
- **False-positive notes:** Audit totals include transitive packages and possibly unreachable paths; do not dismiss without reachability analysis.

16.3 Lifecycle, data-integrity, UI, and operational risks

ID	Priority	Current flaw/risk and evidence	Recommended solution	Verification/closure criterion
RISK-011	P1	Proposal/progress/thesis review services are incomplete: absent assignments or queues, evidence access conflicts, no usable UI, no notifications/release view.	Create version-specific assignments, role queues, evidence access, review forms, admin release, student release view, and notifications.	Full browser E2E for each review type; unauthorized/unreleased access tests pass.
RISK-012	P1	Ethics is document-only but is named/treated as approval and gates thesis merely on existence.	Product decision: implement formal status/reviewer/decision/review/expiry/exemption, or rename the capability and remove false approval claims.	Approved state model and acceptance tests; downstream gates use the approved policy.
RISK-013	P1	Retired supervisor sign-off/review panels remain in schema, routes, labels, docs, and overdue logic; every report ages overdue.	Define reporting cycles/due dates and the authoritative review model; remove retired artifacts and migrate data.	No active 410 links/labels; overdue detects missing/late obligations correctly.
RISK-014	P1	Viva outcome is one shared Examiner-controlled value with no date guard, quorum, independent recommendations, report, comments, actor audit, or ratification.	Store per-Examiner recommendations and report attachments; require scheduled date, quorum/consensus/chair/admin decision, reason, and audit.	Multi-Examiner E2E proves independent inputs, ratification, transitions, and notices.
RISK-015	P1	Submission metadata/notifications precede browser storage completion for progress, thesis, and corrections; public/proposal/ethics uploads can orphan objects.	Staged upload session with finalize/abort, object verification, idempotency key, expiry cleanup, and retry/resume.	Simulated failed/partial upload leaves no final record/notice; retry is idempotent.
RISK-016	P1	Notification coverage/taxonomy is incomplete and misleading; delivery/in-app writes are not atomic; no retry/outbox/dedup/deep links.	Domain-event catalogue plus transactional outbox, worker retries/backoff, idempotency, destination URLs, accurate logs and admin retry tooling.	Trigger matrix is automated; failure/retry/restart tests show exactly-once user-visible intent.
RISK-017	P1	Temporary passwords use <code>Math.random()</code> , are emailed in plaintext, and no forced reset exists.	Firebase password-setup/reset invitation with cryptographic token, expiry, one-time use, and first-login policy; never send reusable password.	No password in email/log; expired/reused invite fails; initial setup flow E2E passes.
RISK-018	P1	Registration can be renewed	Student renewal request +	Concurrency test produces

ID	Priority	Current flaw/risk and evidence	Recommended solution	Verification/closure criterion
		immediately/repeatedly; concurrency can create multiple active periods; graduation/archive leaves active renewal/reminders.	admin approval, eligibility/evidence/payment hooks, idempotency, non-overlap/one-active DB invariant, atomic graduation closure.	one active row; graduated/archived renewal/reminder is blocked.
RISK-019	P1	Finalization/archive is inconsistent across generic status, specialized graduation, Student archive, <code>Thesis.isArchived</code> , registration, user/Firebase access.	One atomic lifecycle command and approved state diagram; separate academic finalization, record retention, and account deactivation.	All related states/audit/notifications commit or roll back together; archive list is accurate.
RISK-020	P2	Status fields overwrite history; assignment deletion removes history; most decisions lack actor/reason/version/time.	Append-only <code>LifecycleTransition</code> /decision/audit records and effective-dated assignments.	Every state/assignment/decision change is attributable and queryable; immutability test passes.
RISK-021	P2	Admin proposal decision is evidence-blind; correction approval lacks file open, reason, rejection, confirmation; application rejection lacks reason.	Evidence-first work queues with document/review panels, required decision notes, confirmation, and audit.	Admin cannot decide without required evidence/notes; UI and audit E2E pass.
RISK-022	P2	Dashboard KPI definitions are misleading: ethics packages called approvals, historical exams called current/upcoming, correction files called cases, final archive counted as archived, all-time failures called current health.	Define each KPI with owner, SQL/filter, time window, status label, drill-down and test; display last updated.	Product-approved metric dictionary matches API and drill-down dataset.
RISK-023	P2	Dead/misnamed navigation: hard-coded profile, missing settings, <code>href="#"</code> , home-loop quick actions, "evaluate/sign" labels for view-only retired work.	Authenticated identity in shell; remove generic arbitrary-role fallback; correct/disable every route/action; rename based on real responsibility.	Automated link crawl and role E2E find no dead/misleading destinations.
RISK-024	P2	Notification UI has no real unread badge, authoritative count, refresh, individual read, pagination, deep link, or failure recovery.	Return unread total/action URL; poll or refresh-on-open; individual/bulk read; view-all; admin resend/retry.	>8 unread scenario shows correct count; deep links and retry flows pass.
RISK-025	P2	Proposal evaluation has no assignment; any non-conflicted Examiner with ID can submit/read all evaluations, while source	Add explicit assignment and least-privilege evaluation access; hide other reviews until permitted; provide version evidence.	Unassigned Examiner receives 403; assigned Examiner sees only permitted evidence/reviews.

ID	Priority	Current flaw/risk and evidence	Recommended solution	Verification/closure criterion
		docs are denied.		
RISK-026	P2	Specialized thesis service denies Supervisor downloads while general repository grants assigned-student thesis access; download audit covers only one path.	Central document authorization policy reused by every download path; product-approved Supervisor policy; unified immutable access audit.	Same authorization result across all paths; every sensitive download creates one audit event.
RISK-027	P2	Correction type is self-selected and not linked to viva result; resubmitting thesis while corrections required bypasses correction approval; one approved package can ignore pending items.	Create correction requirement/order tied to viva/version with type, instructions, due date, reviewer, disposition; block alternate state bypasses.	State-machine tests enforce required order, matching type, all mandatory items resolved.
RISK-028	P2	Application under-review items disappear from current admin list; no applicant tracking/reference/notifications.	Admin work queue across Submitted/Under Review with owner/SLA; public opaque reference and secure status channel; decision reasons/notices.	Under-review remains discoverable; applicant receives correct event without exposing internal data.
RISK-029	P2	No working CI/deploy/migration/E2E pipeline; Playwright is outside <code>npm test</code> ; integration tests mock external services.	Required CI checks with <code>npm ci</code> , audit/SAST, build, migrations, disposable Postgres/Supabase/Firebase emulators, Playwright and accessibility scan.	Protected branch cannot merge/deploy unless all required checks pass.
RISK-030	P2	No scheduler health, backup/restore, retention, privacy/deletion, recovery, or operational runbooks are visible.	Define infrastructure as code, job monitoring, backup/restore drills, retention/legal hold, incident/recovery runbooks, RPO/RTO.	Runtime evidence and successful restore/job-failure drills documented.
RISK-031	P2	Accessibility gaps: missing live regions/labels/skip link, nested mains, motion, 360px minimum, icon label, mobile drawer uncertainty.	Shared accessible patterns; reduced-motion support; semantic landmarks/labels; browser testing at keyboard, 320px, 400% zoom, contrast.	WCAG 2.2 AA audit findings resolved or explicitly accepted with evidence.
RISK-032	P3	Empty route file, production test route, retired models/routes, duplicated student progress route, stale documentation create developer confusion.	Remove/quarantine test/empty/retired artifacts after migration; redirect duplicates; keep one generated route/workflow catalogue.	Route inventory contains no inert/stale surface; docs match tests and state model.
RISK-033	P3	Notification/export/report UX lacks full filtered CSV, trends, latency, ownership,	Server-side full export jobs with authorization/audit; actionable health dashboards	Export row count matches filtered dataset; large exports are safe and auditable.

ID	Priority	Current flaw/risk and evidence	Recommended solution	Verification/closure criterion
		and operational drill-down.	and saved filters.	
RISK-034	P3	Programme branding says MPhil/PhD while code accepts MSc/MEng; terminology such as ethics “approval,” archived thesis, and review “sign-off” conflicts with behavior.	Approve a glossary/product scope; align branding, enum labels, routes, help text, emails, and reports.	Terminology review signed off; automated content/route checks prevent regression.

16.4 Recommended implementation order

107. **Immediate containment:** block/remove /api/auth/claims; exclude review attachments from candidate/supervisor repository; restrict current production exposure; inventory compromised roles/access.
108. **Integrity foundation:** logical multi-file versions, centralized document permissions, staged upload finalization, authoritative role resolution, CSRF/cron hardening.
109. **Policy decisions:** approve ethics, progress-review, viva governance, correction, renewal, finalization/archive, and terminology state diagrams before further UI work.
110. **Workflow completion:** build Examiner/admin review queues and release views, evidence-first decisions, governed viva/corrections, and consistent graduation/registration closure.
111. **Reliable communications:** domain events, outbox/retry/dedup, accurate templates/taxonomy, complete trigger coverage, action links, and notification UX.
112. **Dashboard/UI correction:** fix identity, links, metrics, accessible patterns, then expose reports and operational queues.
113. **Production controls:** dependency upgrades, CI, live integration/E2E, scheduler/monitoring, security headers, upload scanning, backups/retention/runbooks, and accessibility verification.

The project team should not mark a risk “closed” solely because code was merged. Closure requires the stated verification evidence and an updated capability/workflow baseline in this document.